AgroSight: next steps

From a good-looking app with a live scanner to a tool a farmer can trust in the field.

Where we are today

Area	Status	Notes
Farmer UI (home, scan, results, history, crop guide)	Done	Mobile-first, sharp design system, light and dark mode.
Live camera scanner	Built, needs real-device test	Opens camera on phone and laptop, checks light, sharpness, steadiness and leaf coverage, auto-captures. Not yet tested on real phones.
Disease detection	**NOT REAL YET**	The app returns canned example results for every photo. The result page says 'Demo mode'. This is the main gap.
Disease map and statistics	Sample data	All numbers are placeholders. Must be replaced with real scan data or removed.
Urdu / local language	Not done	The earlier language switch was cosmetic and was removed.
Offline use, accounts, sync	Not done	History is stored on the device only.

The one thing that matters most: until a trained model is connected, nothing the app says about a leaf is true. Steps 2 and 3 below are the real work. Everything else is polish around them.

The plan, in order

01 Test the live scanner on real phones (1-2 days)

Browsers only allow camera access on a secure connection. http://localhost works on your laptop, but a phone opening your computer's IP address over http will show 'Camera unavailable'.

- **Fastest:** deploy to Vercel (free). You get an https link to open on any phone.
- **Local alternative:** run `npx next dev --experimental-https`, or use a tunnel such as ngrok or Cloudflare Tunnel.
- Test on at least one Android phone, one iPhone (Safari) and a laptop webcam, in sunlight and indoors.
- Tune the five thresholds at the top of `src/components/scanner/LiveCamera.tsx` (leaf ratio, sharpness, light range, motion, hold time) until auto-capture feels right: not too eager, not stuck.
- Check that the back camera is chosen by default on phones and the flip button works.

Done when: auto-capture fires within about 2 seconds on a steady, well-lit leaf on all three devices, and never fires on an empty wall or a hand.

02 Build the real disease model (the big one, 3-6 weeks)

Detection quality depends almost entirely on the photos you train on. Public datasets are a start, but they are mostly lab-style photos on plain backgrounds and perform badly on real field images.

- **Pick crops and diseases first.** Start small and honest: for example wheat (yellow rust, brown rust, healthy) and cotton (leaf curl virus, bacterial blight, healthy). 3-5 classes per crop beats 38 weak ones.
- **Datasets to start from:** PlantVillage, PlantDoc, and public wheat-rust and cotton leaf sets on Kaggle. Check each licence before use.
- **Collect local field photos.** Even 300-500 real photos per class from Punjab or Sindh fields, taken on cheap phones in mixed light, will lift field accuracy more than any model trick. Have an agronomist label them.
- **Add two extra classes:** 'not a leaf' and 'unclear'. Otherwise the model will confidently name a disease for a hand or a shoe.
- **Train with transfer learning** (MobileNetV3 or EfficientNet-B0 from pretrained weights) with heavy augmentation: brightness, blur, rotation, cluttered backgrounds.
- **Evaluate on field photos only** that the model never saw, per class. Report accuracy and the confusion matrix, not one headline number.
- **Set the confidence gate** (currently 60%) from real results, not by guess.

Done when: on held-out real field photos, each class reaches an agreed target (suggest 85%+ recall for the dangerous diseases), and non-leaf images are rejected.

03 Connect the model to the app (3-5 days)

The front end already expects a backend. The contract is in src/services/ai/modelApiService.ts. You do not need to redesign anything.

- Export the model (ONNX or TorchScript) and wrap it in a small **FastAPI** service with `POST /predict` taking an image and returning crop, disease, confidence and status.
- Keep the treatment advice in a curated table keyed by crop and disease (the library in `diseasesData.ts` is the starting point), reviewed by an agronomist. Do not let the model generate advice.
- Set `NEXT_PUBLIC_AI_PROVIDER=api` and `NEXT_PUBLIC_API_BASE_URL`. Host the API (Render, Railway, Fly.io or a small VPS).
- Remove the silent fallback to the mock in `predictionService.ts`. If the server is down, show 'Can't check right now', never a fake result.
- Remove the 'Demo mode' banner only after this step works end to end.

Done when: scanning a real leaf on your phone returns the real model's answer, and turning the server off shows a clear error.

04 Make the scanner truly live (1-2 weeks)

Today the scanner checks image quality live and then classifies one captured photo. For true real-time labels over the camera view the model has to run on the device.

- Convert the model to run in the browser (onnxruntime-web or TensorFlow.js), 2-4 frames per second.
- Show a live label over the viewfinder and only lock a result when the same answer wins several frames in a row (majority vote). This removes flicker and one-off mistakes.
- Keep the server model as the 'final answer' on capture, and use the small on-device model for the live preview and for offline use.

Done when: the viewfinder shows a steady live label on a mid-range phone without heating up or lagging.

05 Make it farmer-ready (2-3 weeks, can overlap)

A tool can be accurate and still not get used. These are the gaps that decide that.

- **Urdu** (and Punjabi/Sindhi if relevant): real translations, right-to-left layout, reviewed by native speakers. Spoken or icon-based guidance helps low-literacy users.
- **Works offline:** make it an installable PWA with the on-device model cached, and queue scans to sync later. Connectivity in fields is unreliable.
- **Local advice:** link treatments to products and doses legal and available locally, and to a local extension helpline.
- **Replace the stock sample photos** (they show the wrong crops) with real labelled leaves.
- **Disease map:** either feed it real, anonymised scan counts with location consent, or remove it until real data exists.
- **Field test with 5-10 farmers.** Watch them use it without help. Fix whatever they get stuck on.

Done when: five farmers who have never seen it can complete a scan and act on the advice without being told what to do.

06 Safety and trust (ongoing)

A wrong answer here can cost a farmer a crop or lead to the wrong chemical being sprayed.

- Keep the expert review queue: low-confidence scans go to an agronomist, and their corrections become new training data.
- Never show a confident disease name below the gate. Keep the 'Not sure, try again' path.
- Keep spray safety notes and 'confirm with your agriculture officer' on every result.
- Track real-world accuracy after launch, and retrain on a schedule.

Done when: there is a monthly report of real accuracy, and corrected scans flow back into training.

Rough timeline

Step	Effort	Depends on	Who
01 Real-device scanner test	1-2 days	Nothing	You + me
02 Real disease model	3-6 weeks	Chosen crops, field photos, an agronomist	ML work + agronomist
03 Connect model to app	3-5 days	Step 02 model exported	Me

Step	Effort	Depends on	Who
04 On-device live labels	1-2 weeks	Step 02 model	ML + me
05 Farmer-ready features	2-3 weeks	Can start in parallel	Me + translators
06 Safety loop	Ongoing	Steps 02-03	Everyone

What I would do first

- **This week:** deploy to Vercel and test the scanner on phones (Step 01). Decide the first 2 crops and 3-5 diseases each.
- **In parallel:** start collecting and labelling real field photos. This is the slowest part and the one that cannot be shortcut.
- **Then:** train a first small model, wire it in (Steps 02-03), and run a small field test before adding anything else.

Open questions for you

- Which crops and diseases matter most for your first users?
- Do you have access to an agronomist to label photos and review advice?
- Does the app need to work with no internet at all, or is a weak connection fine?
- Is this for a demo or expo, or for real farmers? The answer changes how much of Steps 04-06 is needed now.